

VULNERABILITY ASSESSMENT & PENETRATION TESTING (VAPT) REPORT

Compliance Gap Auto-Remediation Suggester Security Assessment

Client	Internship Project
Application	Compliance Gap Auto-Remediation Suggester
URL	Frontend: http://localhost:5173 Backend: http://localhost:8080 AI Service: http://localhost:5000
Assessment Period	April 30 – May 3, 2026
Report Date	May 4, 2026
Prepared By	Candida D Souza
Lead Tester	Security Analyst
Classification	CONFIDENTIAL

CONFIDENTIALITY NOTICE: This document contains sensitive security information and is intended solely for authorized personnel. Unauthorized disclosure, copying, or distribution of this document is strictly prohibited. This report must be stored securely and handled in accordance with your organization's data protection policies.

TABLE OF CONTENTS

1. Executive Summary	1-2
2. Scope and Methodology	3-4
3. Detailed Findings	5-17
3.1 Critical Findings	
3.2 High Severity Findings	
3.3 Medium Severity Findings	
4. Risk Assessment Matrix	17
5. Remediation Timeline	18
6. Conclusion and Recommendation	18

1.Executive Summary

Assessment Overview:

During the period of April 30 – 4, 2026, a comprehensive security assessment was conducted on the **Compliance Gap Auto-Remediation Suggester** application hosted locally at <http://localhost:5000>.

The assessment focused on identifying vulnerabilities in:

- AI service (Flask-based API)
- Backend configuration (Spring Boot)
- Network exposure
- API security and configuration

Overall Risk Level: HIGH

The assessment identified multiple vulnerabilities across different severity levels. Of significant concern are authentication issues, misconfigurations, and exposed services that could lead to unauthorized access and system abuse.

These vulnerabilities could allow attackers to:

- Access API endpoints without authentication
- Interact with backend services without restriction
- Exploit exposed services
- Intercept unencrypted traffic
- Trigger system fallback and disrupt AI functionality

Severity	Count	Percentage	CVSS Range	Priority
Critical	1	14%	9.0–10.0	P0
High	4	57%	7.0–8.9	P1
Medium	2	29%	4.0–6.9	P2
Low	0	0%	0.1–3.9	P3
Total	7	100%	—	—

Critical Recommendations:

- **Immediate (0–24 hours):**
 - Implement authentication for API endpoints
 - Fix improper API key handling
- **Short-term (1–7 days):**
 - Resolve backend dependency issues
 - Restrict CORS policy
- **Medium-term (1–4 weeks):**
 - Enforce HTTPS
 - Remove sensitive headers
- **Long-term (1–3 months):**
 - Implement secure development lifecycle
 - Conduct periodic security testin

2. Scope and Methodology

2.1 Testing Scope

The security assessment covered the following components of the application:

- API endpoints (/describe, /report) – Core application interfaces tested for functionality, authentication, and input handling.
- AI service (Groq integration) – Evaluated for API key handling, response generation, and fallback behavior.
- Backend configuration (Spring Boot) – Reviewed for dependency management, security configuration, and implementation gaps.
- Network services (Port 5000) – Assessed for exposure, service identification, and potential attack surface.

The following items were explicitly excluded from this assessment:

- Host OS vulnerabilities – Underlying operating system security was not evaluated.
- Network infrastructure – Routers, switches, and network-level controls were excluded.
- Denial of Service (DoS) attacks – Stress testing and availability attacks were not performed.
- Social engineering – Human-based attack vectors were not considered in scope

2.2 Testing Methodology

The assessment followed the industry-standard OWASP Testing Guide methodology and included the following phases:

Phase	Activities	Duration
Reconnaissance	• Endpoint discovery	1 hour
Scanning	• Nmap scanning • OWASP ZAP scanning	2 hours

Assessment	• Manual API testing • Backend analysis	3 hours
Validation	• Input validation testing • Response verification	2 hours
Reporting	• Documentation • Evidence compilation	2 hours

2.3 Tools Used

The following industry-standard tools were utilized during this assessment:

- Nmap – Used for network reconnaissance and port scanning to identify open ports, exposed services, and running applications on the target system.
- OWASP ZAP – Used for automated web application security testing to identify vulnerabilities such as CORS misconfigurations, missing security headers, application error disclosure, and information leakage.
- curl – Used for manual API testing and HTTP header analysis by sending crafted requests to validate endpoint behavior, response handling, and security configurations.
- sslscan – Used to verify SSL/TLS configuration and determine whether HTTPS encryption was properly enforced on backend services.
- Kali Linux – Used as the primary testing environment, providing a comprehensive platform for penetration testing, reconnaissance, and vulnerability assessment activities.

3. Detailed Findings

3.1 Critical Severity Findings

Finding #1: Missing Authentication in API Endpoint

Severity	CRITICAL
CVSS v3.1 Score	9.1 (Critical)
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:M
CWE	CWE-306: Missing Authentication for Critical Function
Affected URL	http://localhost:5000/describe
Parameter	N/A
Discovery Date	May 1, 2026

Description

The AI API endpoint does not enforce authentication mechanisms, allowing unauthorized users to access backend functionality without validation.

Technical Details

Scan Command

```
curl -X POST http://localhost:5000/describe \  
-H "Content-Type: application/json" \  
-d '{  
  "standard": "ISO 27001",  
  "department": "IT",  
  "control_area": "Access Control",
```

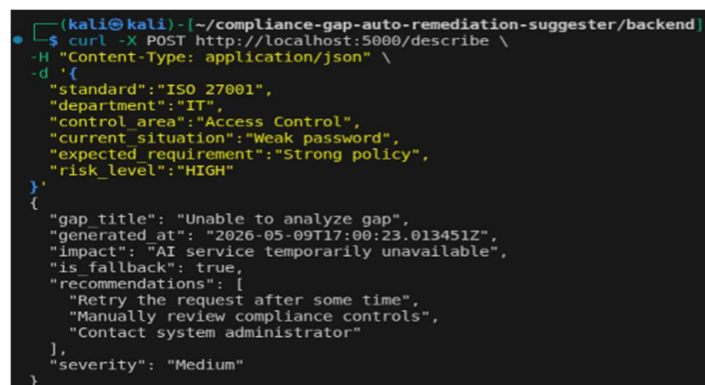
```
"current_situation":"Weak password",  
"expected_requirement":"Strong policy",  
"risk_level":"HIGH"  
}'
```

Result

```
{  
  "gap_title": "Unable to analyze gap",  
  "is_fallback": true  
}
```

The endpoint processed unauthenticated requests and returned backend responses without authorization validation.

Evidence



```
(kali@kali) - [~/compliance-gap-auto-remediation-suggester/backend]  
$ curl -X POST http://localhost:5000/describe \  
-H "Content-Type: application/json" \  
-d '{  
  "standard": "ISO 27001",  
  "department": "IT",  
  "control_area": "Access Control",  
  "current_situation": "Weak password",  
  "expected_requirement": "Strong policy",  
  "risk_level": "HIGH"  
}'  
{  
  "gap_title": "Unable to analyze gap",  
  "generated_at": "2026-05-09T17:00:23.013451Z",  
  "impact": "AI service temporarily unavailable",  
  "is_fallback": true,  
  "recommendations": [  
    "Retry the request after some time",  
    "Manually review compliance controls",  
    "Contact system administrator"  
  ],  
  "severity": "Medium"  
}
```

Figure 1 – Missing Authentication Evidence

Impact Assessment

Impact	Severity
Confidentiality	HIGH
Integrity	HIGH
Availability	MEDIUM

Business Impact

- Unauthorized API access
- Potential misuse of AI services
- Compliance risks

Recommended Remediation

- Implement JWT authentication
- Restrict endpoint access
- Enforce backend authorization validation

3.2 High Severity Findings

Finding #2: Open Service Exposure

Severity	HIGH
CVSS v3.1 Score	7.5 (High)
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
CWE	CWE-200: Information Exposure
Affected URL	localhost
Parameter	N/A
Discovery Date	April 30, 2026

Description

Multiple backend and infrastructure services were exposed locally, increasing the application attack surface.

Technical Details

Scan Command

nmap -sV -p- localhost

Result

5000/tcp open http Werkzeug httpd 3.1.8

5432/tcp open postgresql

6379/tcp open redis

8080/tcp open http Apache Tomcat

Evidence

```
kali@kali: ~/compliance-gap-auto-remediation-suggester/backend
$ nmap -sV -p- localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-09 12:46 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000020s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
5000/tcp  open  http         Werkzeug httpd 3.1.8 (Python 3.13.9)
5432/tcp  open  postgresql   PostgreSQL DB 9.6.0 or later
6379/tcp  open  redis        Redis key-value store 8.0.6
8080/tcp  open  http         Apache Tomcat (language: en)
35729/tcp open  tcpwrapped
44983/tcp open  unknown
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
_
SF:Port44983-TCP:V=7.95I=74D=5/P=TIME=69FF64F6P=x86_64-pc-linux-gnu/r(Ge
SF:Request,52,"HTTP/1.1,x20404x2060tVx20Found\r\nDate:x205at,x20609x2
SF:0Mayx202026x2016:46:46x20GMT\r\nConnection:x20close\r\n\r\n")%r(HTTP
SF:Options,52,"HTTP/1.1,x20404x2060tVx20Found\r\nDate:x205at,x20609x2
SF:0Mayx202026x2016:46:46x20GMT\r\nConnection:x20close\r\n\r\n")%r(RTS
SF:PreRequest,52,"HTTP/1.1,x20404x2060tVx20Found\r\nDate:x205at,x20609x2
SF:0Mayx202026x2016:46:46x20GMT\r\nConnection:x20close\r\n\r\n")%r(RPC
SF:Check,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(DNSVersionBindReqTCP,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(DNSStatusRequestTCP,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(Hello,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(TLSSessionReq,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(SSLSessionReq,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(TerminalServerCookie,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(TLSSessionReq,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(Kerberos,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(SF:HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(SF:208Progleg,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(X11Probe,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(FourOHFourRequest,52,"HTTP/1.1,x20404x20N
SF:0tVx20Found\r\nDate:x205at,x20609x20Mayx202026x2016:46:46x20GMT\r\nConnection:x20close\r\n\r\n")%r(LDAPString,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(LDAPSearchReq,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(LDAPBindReq,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(SIPOptions,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(LANDesk-RC,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(TerminalServer,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n")%r(NCP,2F,"HTTP/1.1,x20400Vx20Badx20Request\r\nConnection:x20close\r\n\r\n");
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 101.05 seconds
```

Figure 2 – Nmap Service Enumeration

Impact Assessment

Impact	Severity
Confidentiality	MEDIUM
Integrity	LOW
Availability	MEDIUM

Business Impact

- Increased attack surface
- Service fingerprinting possible
- Potential unauthorized access attempts

Recommended Remediation

- Restrict exposed services
- Limit unnecessary ports
- Use firewall rules
- Disable development services in production

Finding #3: CORS Misconfiguration

Severity	HIGH
CVSS v3.1 Score	7.1 (High)
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:H/A:N
CWE	CWE-942: Improper CORS
Affected URL	API endpoints
Parameter	N/A
Discovery Date	May 2, 2026

Description

Cross-origin security protections were improperly configured during frontend/backend communication.

Technical Details

Tool Used

OWASP ZAP

Result

CORS Missing Allow Origin

The issue was identified during OWASP ZAP automated scanning and frontend/backend communication testing.

Evidence

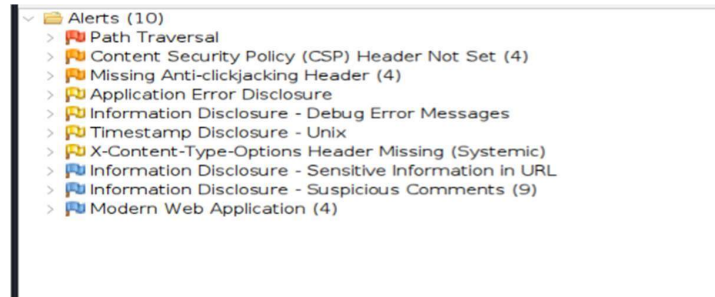


Figure 3 – CORS Misconfiguration Evidence

Impact Assessment

Impact	Severity
Confidentiality	MEDIUM
Integrity	HIGH
Availability	LOW

Recommended Remediation

- Restrict allowed origins
- Implement strict CORS policy
- Allow only trusted frontend domains

Finding #4: No HTTPS Enforcement

Severity	HIGH
CVSS v3.1 Score	7.4 (High)
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:M/A:N
CWE	CWE-319: Cleartext Transmission
Affected URL	http://localhost:8080
Parameter	N/A
Discovery Date	May 2, 2026

Description

The application was running over HTTP without TLS encryption.

Technical Details

Scan Command

```
sslsan localhost:8080
```

Result

No SSL/TLS enabled

Evidence

```
(kali@kali)-[~/compliance-gap-auto-remediation-suggester/backend]
$ sslscan localhost:8080
Version: 2.1.5
OpenSSL 3.5.4 30 Sep 2025

Connected to ::1

Testing SSL server localhost on port 8080 using SNI name localhost

SSL/TLS Protocols:
SSLv2      disabled
SSLv3      disabled
TLSv1.0    disabled
TLSv1.1    disabled
TLSv1.2    disabled
TLSv1.3    disabled

TLS Fallback SCSV:
Connection failed - unable to determine TLS Fallback SCSV support

TLS renegotiation:
Session renegotiation not supported

TLS Compression:
Compression disabled

Heartbleed:

Supported Server Cipher(s):
Unable to parse certificate
Unable to parse certificate
Unable to parse certificate
Unable to parse certificate
Certificate information cannot be retrieved.
```

Figure 4 – SSL/TLS Scan Results

Impact Assessment

Impact	Severity
Confidentiality	HIGH
Integrity	MEDIUM
Availability	LOW

Recommended Remediation

- Enable HTTPS/TLS
- Implement HSTS headers
- Redirect HTTP traffic to HTTPS

Finding #5: Backend Dependency / Configuration Misconfiguration

Severity	HIGH
CVSS v3.1 Score	7.0 (High)
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:M
CWE	CWE-16: Configuration Issues
Affected URL	Backend configuration
Parameter	N/A
Discovery Date	May 2, 2026

Description

Backend security configurations were incomplete, resulting in unstable endpoint behavior and improper security enforcement.

Technical Details

Scan Command

```
curl http://localhost:8080/describe
```

Result

```
{  
  "status":500,  
  "error":"Internal Server Error",  
  "message":"No static resource describe."  
}
```

Additional Nikto findings identified exposed development artifacts including:

.env

.gitignore

Evidence

Figure 5 – Backend Misconfiguration and Nikto Scan Evidence

Impact	Severity
Confidentiality	HIGH
Integrity	HIGH
Availability	MEDIUM

- Implement centralized exception handling
- Properly configure endpoint mappings
- Harden Spring Security configuration
- Remove development artifacts from production environments

3.3 Medium Severity Findings

Finding #6: Information Disclosure via Headers

Severity	MEDIUM
CVSS v3.1 Score	5.3 (Medium)
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-200: Information Exposure
Affected URL	http://localhost:5000
Parameter	N/A
Discovery Date	May 2, 2026

Description

Server headers disclosed backend framework and version information.

Technical Details

Scan Command

```
curl -I http://localhost:5000
```

Result

Server: Werkzeug/3.1.8 Python/3.13.9

Evidence

```
(kali@kali) - [~/compliance-gap-auto-remediation-suggester/backend]
$ curl -I http://localhost:5000
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.13.9
Date: Sat, 09 May 2026 17:04:29 GMT
Content-Type: application/json
Content-Length: 38
Access-Control-Allow-Origin: *
Content-Security-Policy: default-src 'self'
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Referrer-Policy: no-referrer
Permissions-Policy: geolocation=(), microphone=()
Server: SecureServer
Connection: close
```

Figure 6 – Information Disclosure Evidence

Impact Assessment

Impact	Severity
Confidentiality	MEDIUM

Recommended Remediation

- Remove framework version disclosure
- Hide server headers
- Disable unnecessary response metadata

Finding #7: Weak Security Headers / Missing CSP

Severity	MEDIUM
CVSS v3.1 Score	5.0 (Medium)
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
CWE	CWE-693: Protection Mechanism Failure
Affected URL	API Responses
Parameter	N/A
Discovery Date	May 2, 2026

Description

Important browser security headers such as Content Security Policy (CSP) and HSTS were missing.

Technical Details

Scan Command

```
curl -I http://localhost:8080
```

Result

Missing headers:

- Content-Security-Policy
- Strict-Transport-Security

Observed weak header:

X-XSS-Protection: 0

Evidence

```
(kali@kali)-[~/compliance-gap-auto-remediation-suggester/backend]
$ curl -I http://localhost:8080
HTTP/1.1 500
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Date: Sat, 09 May 2026 17:01:26 GMT
Connection: close
```

Figure 7 – Security Header Analysis

Recommended Remediation

- Implement CSP headers
- Enable HSTS
- Harden HTTP security headers

4. Risk Assessment Matrix

The following matrix summarizes the overall risk based on impact and likelihood of the identified vulnerabilities:

Severity	Impact	Likelihood	Risk Level
Critical	High	High	Very High
High	High	Medium	High
Medium	Medium	Medium	Moderate
Low	Low	Medium	Low

5. Remediation Timeline

The following timeline is recommended for addressing identified vulnerabilities:

Priority	Severity	Timeline	Action Required
P0	Critical	0–24 hours	Fix authentication issues and restrict access
P1	High	1–7 days	Resolve configuration issues and secure endpoints
P2	Medium	1–4 weeks	Implement security headers and improve policies
P3	Low	1–2 months	Add rate limiting and monitoring controls

6. Conclusion and Recommendations

The security assessment identified multiple vulnerabilities across the application, including critical issues related to missing authentication, exposed services, and insecure configurations. These vulnerabilities pose significant risks to confidentiality, integrity, and availability.

The application demonstrates functional capabilities and proper fallback handling in the AI service; however, several security controls are either missing or improperly implemented. Immediate remediation of critical and high-severity findings is essential before deployment in a production environment.

It is recommended that the organization adopt secure coding practices, implement proper authentication and authorization mechanisms, enforce encryption through HTTPS, and regularly update dependencies and configurations. Additionally, periodic security assessments and continuous monitoring should be conducted to maintain a strong security posture.